Miguel dos Santos Neto (125200) Diogo Coelho e Silva (125240)

MPEI: Previsões Meteorológicas

1. Introdução

A previsão meteorológica desempenha um papel fundamental no nosso quotidiano, influenciando desde as decisões mais simples do dia a dia até ao planeamento estratégico em setores vitais como a agricultura, a logística e a gestão de energia. Com o constante crescimento do volume de dados climáticos gerados diariamente, a aplicação de ferramentas computacionais inteligentes tornou-se indispensável para transformar estes registos em previsões úteis e compreensíveis.

Este projeto nasce com o objetivo de aplicar, num cenário prático e real, as diferentes metodologias lecionadas ao longo da unidade curricular: o **Naive-Bayes**, o **Bloom Filter** e o **MinHash**. Em vez de olhar para estas ferramentas apenas de uma perspetiva teórica, o foco do trabalho centra-se em perceber como diferentes abordagens de análise de dados se podem comportar e complementar na resolução de um problema tão dinâmico como o clima.

2. Processamento de Dados

Os dados necessários para o treino e teste do sistema foram obtidos através da Open-Meteo, uma API *open-source* que disponibiliza dados meteorológicos. Adicionalmente, foi desenvolvido um programa com o objetivo de normalizar as diferentes categorias meteorológicas presentes nos dados. Por exemplo, várias categorias distintas de aguaceiros foram agregadas numa única categoria através dos WMO Codes dados pela API, de modo a garantir maior consistência e simplificar o processo de análise e modelação.

Uma vez que as estruturas MinHash e Bloom Filter não operam diretamente sobre valores numéricos contínuos, foi necessário proceder à sua discretização e representação sob a forma de *tokens* categóricos. Para esse efeito, foi implementado o módulo extract_features.m, responsável pela conversão de atributos numéricos em etiquetas descritivas (por exemplo, temp_alta). Os limites (*thresholds*) utilizados nesta transformação foram previamente calculados através do módulo compute_thresholds.m, permitindo segmentar os valores numéricos em intervalos representativos e adequados ao processamento pelas estruturas de dados utilizadas.

3. Naive-Bayes

O classificador Naive-Bayes Gaussiano foi implementado em duas partes: treino e predição.

Na parte de treino (nb_train.m), o modelo estima, para cada classe meteorológica (sun, drizzle, rain, thunder, fog e snow), a média e desvio-padrão de cada atributo a partir das amostras de treino correspondentes. É também calculado o logaritmo da probabilidade *a priori* de cada classe com base na sua frequência relativa. O desvio-padrão inclui uma pequena constante (1×10^{-9}) para evitar divisões por zero em features com variância nula.

Na predição (nb_predict.m), aplica-se a regra de Bayes assumindo independência condicional entre *features*. Para cada amostra de teste, calcula-se o logaritmo da probabilidade *a posteriori* de cada classe com a seguinte fórmula:

$$log\ P(c|x)\ \propto\ log\ P(c)\ \ +\ \ \sum_{j} log\ N(x_j|\mu_{c,j}, \sigma_{c,j})$$

Ou seja, o logaritmo da probabilidade de uma classe dado um conjunto de observações é proporcional ao logaritmo da probabilidade a priori dessa classe mais a soma dos logaritmos da densidade gaussiana de cada atributo dado a essa classe.

O cálculo é feito em espaço logarítmico para evitar que as probabilidades arredondam para zero, com normalização por subtração do máximo antes de exponenciar. A classe com maior probabilidade normalizada é retornada como previsão.

4. Bloom Filter

O Bloom Filter tem a responsabilidade de detectar novidades, isto é, padrões de condições atmosféricas nunca antes vistas.

Os tokens das variáveis críticas são concatenados numa única string longa por dia (ex: "temp_alta_hum_media"), servindo de chave para detectar tais padrões.

Uma vez que o Bloom Filter garante a ausência absoluta de falsos negativos, podemos tomar partido disso para a implementação no *pipeline*, que iremos abordar de seguida.

Por outro lado, o filtro admite falsos positivos, contudo o impacto desta falha é controlado: na pior das hipóteses, o modelo simplesmente ignorará o ajuste de novidade e confiará nas probabilidades padrão do Naive-Bayes.

Para a implementação prática, fixou-se o tamanho do vetor de bits em $m\ =\ 50.000$ posições. O número ideal de funções de dispersão (k) a aplicar foi determinado dinamicamente através da fórmula do k ótimo:

$$k\ =\ (m/n)\ *\ ln\,2$$

Onde:

- m é o tamanho do vetor;
- n é o número de amostras do conjunto de treino introduzido no filtro.

Substituindo os valores, obteve-se $k\ \approx\ 4.49$. Como o número de funções de dispersão deve ser inteiro, utilizaram-se 4.

Obteve-se uma taxa de falsos positivos de aproximadamente 4.5%, com recurso à seguinte fórmula teórica:

$$p\ \approx (1\ -e^{-km/n})^{k}$$

Para otimizar o desempenho computacional e evitar o custo de correr quatro algoritmos de hash independentes sobre strings de texto, a função bloom_hash.m recorre à técnica de Double Hashing.

O algoritmo executa apenas duas funções base de 32-bits sobre a string:

- **DJB2**: Algoritmo focado em velocidade de processamento, gerando o primeiro hash base (h_1).
- **FNV-1a**: Algoritmo de alta dispersão, gerando o segundo hash base (h_2).

A partir destes dois valores base, as $k = 4$ posições virtuais necessárias para mapear o vetor de bits são geradas através de uma combinação linear vetorizada, calculada pela fórmula:

$$pos = (h_1 + i * h_2) \, (mod \, m) + 1, \; i \in \{0, 1, 2, 3\}$$

Esta abordagem permite simular o comportamento de 4 funções independentes com o custo computacional de apenas duas.

5. MinHash

O algoritmo MinHash foi integrado no sistema com o objetivo de calcular, de forma computacionalmente eficiente, a proximidade climática entre as amostras.

Os *tokens* são mantidos como um conjunto independente de elementos para que o algoritmo possa calcular as suas permutações e estimar a similaridade de Jaccard entre dias.

O sistema foi configurado para utilizar 200 funções de dispersão independentes para gerar as assinaturas, o que permite obter um compromisso entre a precisão da estimativa e o custo computacional.

6. Pipeline

O principal desafio deste projeto consistiu na integração dos diferentes módulos de classificação numa única arquitetura de decisão. A contribuição de cada módulo é ajustada dinamicamente em função da confiança das previsões produzidas.

Para quantificar essa confiança, foi utilizada a Entropia de Shannon, uma métrica que mede o grau de incerteza associado a uma distribuição de probabilidades. A sua definição é dada por:

$$- \sum_{i=1}^{n} P(x_i) \, log_2 \, P(x_i)$$

Onde $P(x_i)$ representa a probabilidade de ocorrência da categoria x_i.

- **Entropia Máxima**: Ocorre quando todas as categorias têm exatamente a mesma probabilidade de acontecer. O sistema está em máxima incerteza.
- **Entropia Mínima**: Ocorre quando um resultado é absolutamente certo.

Esta métrica foi posteriormente utilizada para ajustar dinamicamente os pesos da previsão (ensemble_predict.m), atribuindo maior importância ao Naive-Bayes quando a sua confiança era elevada e aumentando o contributo do MinHash em situações de maior ambiguidade.

7. Execução dos programas

Há 5 programas diferentes para a execução deste sistema, mais um auxiliar:

- **Programas de teste**: há 3 diferentes programas para testar cada um dos componentes individualmente, todos do tipo test_<componente>.m. Dentro de cada, no código, pode alterar o dataset usado na execução (por defeito está o de Beijing), tal como a seed do RNG.
- **Programas de demonstração**: há 2 programas diferentes para a execução da pipeline:
    - main_demo.m é que corre a pipeline completa em modo de linha de comandos; no final imprime todas as métricas de avaliação. O dataset e a seed do RNG podem ser alterados nas primeiras linhas do script.
    - weather_gui.m é a interface gráfica interativa. Permite selecionar o CSV e o rácio de treino/teste através de um slider, treinar o modelo e depois prever a classe meteorológica de uma amostra introduzida manualmente via sliders e campos numéricos para cada feature.
- **Programa auxiliar para geração do dataset**: como mencionado anteriormente, há um programa próprio para transformar os CSVS dados diretamente pelo Open Meteo num simplificado. Ele está localizado em data_srcs, e tem duas variáveis no início para especificar ficheiros de input e output

8. Resultados

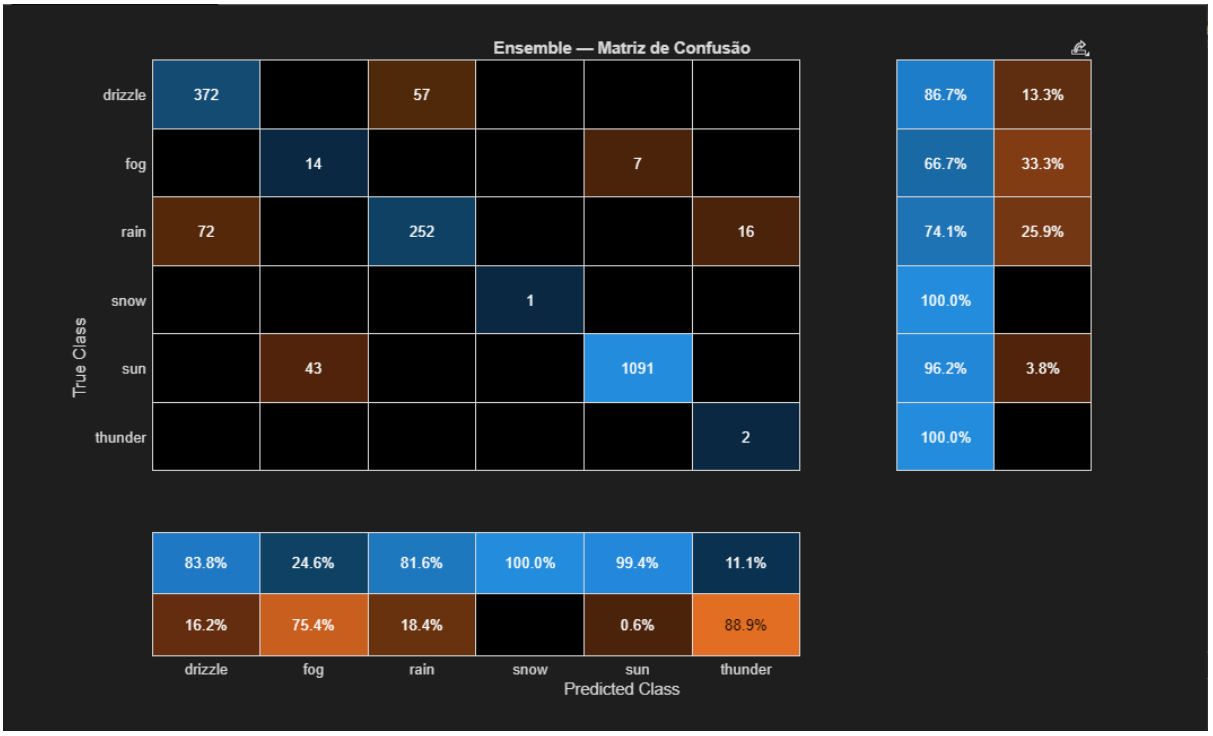
Consideremos o dataset *aveiro.csv* para esta análise.
Os dados foram divididos em 80% treino / 20% teste, permitindo que os diversos módulos tenham informação suficiente para tomar decisões acertadas. O Bloom Filter não deteta dias novos, uma vez que o dataset abrange todas as combinações possíveis.

Comparamos os resultados do Naive-Bayes apenas com os resultados da *pipeline*, de modo a podermos ver a influência dos restantes módulos na decisão final:

Método	**Precisão**
Naive-Bayes	89.62%
Pipeline	89.88%

Note-se que os resultados diferem por poucas décimas devido à entropia, contudo, a pipeline contribui de modo positivo.

Obtemos a seguinte matriz de confusão:



O principal ponto fraco do nosso sistema são os vários tipos de chuva, uma vez que o dataset não fornece os campos necessários para uma boa distinção. Foi por esse motivo que foi escolhido o dataset de aveiro para esta simulação, uma vez que se pode notar essa influência.

Com os restantes datasets, obtém-se uma média de 90% de precisão e o programa corre em cerca de 4 segundos (incluindo o tempo que o motor gráfico leva a gerar as imagens).

Para resultados individuais sobre cada componente, recomenda-se executar os programas de testes que foram providenciados. Podemos afirmar que todos os testes realizados passam ou têm resultados positivos, independentemente das configurações iniciais utilizadas (ficheiro CSV e seed do RNG).

9. Conclusão

Este trabalho demonstrou que a combinação de abordagens probabilísticas distintas permite construir um sistema de previsão meteorológico robusto e adaptativo. O Naive-Bayes destacou-se como componente central da pipeline, o Bloom Filter permitiu identificar padrões atmosféricos inéditos de forma eficiente, e o MinHash acrescentou o sistema de análise de similaridade entre observações. Em conjunto, estas técnicas ilustram como metodologias probabilísticas, habitualmente estudadas isoladamente, se podem integrar num sistema coeso com aplicação prática real, estando o caminho aberto para melhoria futura.